

MDP-DSL: A Compiled Domain-Specific Language for Sequential Decision-Making with Formal Verification and Failure Diagnosis

Charvit Rajani

Department of Electronics and Communication Engineering
Indian Institute of Technology Guwahati
Guwahati 781039, Assam, India
Roll No. 240102028

Abstract—We present MDP-DSL, a compiled domain-specific language for Markov decision processes and their partially observable extensions. The language is implemented as a single-translation-unit C++17 compiler with no external dependencies, organized as a five-phase pipeline: preprocessor, parser, validator with 25 semantic checks, solver, and verifier. Four solver backends are integrated: value iteration, policy iteration, Q-learning, and point-based value iteration (PBVI). A Baum-Welch hidden Markov model fitter bridges learned regime transitions into MDP dynamics as a first-class language construct. The compiler exposes a `VERIFY` block with compile-time exit-code semantics suitable for continuous integration, and a six-class autopsy engine that diagnoses reward misspecification (myopia, hacking, discount cliffs, magnitude imbalance, dead states, and policy fragility) with single-parameter repair suggestions. A separate `mdp_autopsy` binary consumes execution logs and reports model-vs-reality divergence with graded verdicts. We demonstrate the system on the canonical Tiger POMDP, a 4×3 grid-world, an HMM-bridged portfolio optimizer for Indian equity markets, and three failure-analysis demonstrations. The full implementation comprises approximately 13,000 lines of code, and 297 test assertions pass under a single `g++ -O2 -std=c++17` invocation. The intent of this work is to explore whether a compiler-first approach to RL specification can usefully complement the library-based tooling that dominates the current ecosystem.

Index Terms—Markov decision processes, POMDPs, domain-specific languages, formal verification, reward specification, hidden Markov models, point-based value iteration, compiler design.

I. INTRODUCTION

Reinforcement-learning toolchains in widespread use, such as Gymnasium, Stable-Baselines3, and RLlib, optimize the training-and-deployment loop. They are excellent for prototyping neural-network policies. They are not designed, however, for what we argue is the dominant cost in applied RL: the model itself (the transition kernel, the reward shape, the discount factor, the safety properties) is buried inside Python class hierarchies and tensor manipulations. There is no canonical artifact describing what was assumed, diff-able between commits, formally checkable by tooling.

Three observations motivated this work.

First, misspecification is an expensive failure mode. An RL algorithm that converges to a policy satisfying the reward func-

tion in a way the designer did not intend, that is, specification gaming, reward hacking, and discount-cliff phase transitions, is folklore in the community [1]. These failures live in blog posts and conference talks, not in tooling. If the specification were a language a compiler could read, the compiler could complain at edit time.

Second, verification is the natural endpoint of declarativity. Hardware description languages have type-checking; database query languages have query planners. The sequential-decision world has no comparable artifact: no `VERIFY` block at the end of the file. Formal-verification tools for probabilistic systems (PRISM, STORM) exist but check properties of pre-specified policies; they do not themselves solve Bellman optimality.

Third, data and decisions live in different vocabularies. HMMs learn latent regime structure from data [3]; MDPs solve optimal behavior given dynamics [2]. A domain expert who wants to write “fit an HMM on this data file, then plumb its transition matrix into this MDP” has no concise way to express the connection. This work makes that connection a first-class language affordance.

MDP-DSL is offered as a small contribution toward all three. The specific deliverables are: a compiled DSL with 25-rule semantic validation (Section II); four interchangeable solver backends including PBVI for POMDPs (Section IV); a Baum-Welch HMM fitter bridged into the MDP language at compile time (Section V); and an autopsy engine for six classes of reward misspecification plus a backwards-solver binary for model-vs-reality divergence (Section VI). We validate the system on canonical benchmarks and an HMM-bridged portfolio optimizer (Sections VII).

II. LANGUAGE REFERENCE

This section is a complete reference for the surface syntax. A program is a sequence of declarations, one per line, with a leading keyword in upper case. Tokens are whitespace-separated; the `#` character introduces a line comment. Keywords are case-insensitive. No nested scopes or expressions are supported by design; users who want arithmetic in reward declarations are expected to precompute the values.

A. Core MDP declarations

```
STATE: <name>
ACTION: <name>
TRANSITION: <state> <action> <state> <probability>
REWARD: <state> <value>
ACTION_REWARD: <state> <action> <value>
DISCOUNT: <value> # gamma in [0, 1)
SOLVER: vi | pi | ql | pbvi
```

Probabilities must lie in $[0, 1]$. For every (s, a) pair, the declared transitions must sum to 1.0 within 10^{-9} . Either REWARD or ACTION_REWARD may be used (the latter generalizes the former); they may be mixed.

B. POMDP extensions

```
OBSERVATION: <name>
OBSERVE_PROB: <state> <action> <obs> <probability>
INITIAL_BELIEF: <state> <probability>
BELIEF_STATE: <name> <state>:<prob> <state>:<prob> ...
ALPHA_VECTORS: <int> # PBVI parameter (default 15)
PBVI_ITERATIONS: <int> # default 100
HORIZON: <int> # finite-horizon POMDP
```

When SOLVER: pbvi, at least one INITIAL_BELIEF or BELIEF_STATE must be declared. BELIEF_STATE entries provide named beliefs that can be referenced in VERIFY assertions.

C. HMM bridge

```
HMM_STATES: <name> <name> ...
EMISSION_TYPE: gaussian | discrete
FIT_HMM: <csv_path> iterations=<int>
HMM_SEED: <int> # for reproducibility
BRIDGE: hmm -> mdp
```

The bridge runs Baum-Welch on the referenced CSV before validation; the learned transition matrix is plumbed into the AST as TRANSITION: rows, and the HMM states are added to the declared state set. User-specified transitions for the same triples are preserved.

D. Verification

```
VERIFY: pi(<state_or_belief>) == <action>
VERIFY: pi(<state_or_belief>) != <action>
VERIFY: V(<state_or_belief>) > <number>
VERIFY: V(<state_or_belief>) < <number>
VERIFY: V(<state_or_belief>) > V(<state_or_belief>)
```

Both ASCII ($\pi(s)$) and Unicode ($\pi(s)$) forms are accepted. The compiler exits with status 1 if any assertion fails, 0 otherwise, which is intended to fit into CI pipelines.

E. Macros

```
GRID: <rows>x<cols>:<spec_strings>
```

The GRID: macro expands into the equivalent STATE, TRANSITION, and REWARD declarations for a rectangular grid world, with optional walls, terminal cells, and slip probability.

F. Complete example: Tiger POMDP

Listing 1 is the canonical POMDP benchmark expressed in the language. The compiler solves this with PBVI in 100 backups, producing 4 useful alpha-vectors after witness pruning, and both VERIFY assertions pass at exit code 0.

Listing 1. Tiger POMDP in MDP-DSL.

```
STATE: TigerLeft
STATE: TigerRight

ACTION: Listen
ACTION: OpenLeft
ACTION: OpenRight

TRANSITION: TigerLeft Listen TigerLeft 1.0
TRANSITION: TigerRight Listen TigerRight 1.0
# (reset transitions for open actions omitted)

OBSERVATION: HearLeft
OBSERVATION: HearRight

OBSERVE_PROB: TigerLeft Listen HearLeft 0.85
OBSERVE_PROB: TigerLeft Listen HearRight 0.15
OBSERVE_PROB: TigerRight Listen HearLeft 0.15
OBSERVE_PROB: TigerRight Listen HearRight 0.85

ACTION_REWARD: TigerLeft OpenLeft -100.0
ACTION_REWARD: TigerRight OpenLeft 10.0
ACTION_REWARD: TigerLeft OpenRight 10.0
ACTION_REWARD: TigerRight OpenRight -100.0
ACTION_REWARD: TigerLeft Listen -1.0
ACTION_REWARD: TigerRight Listen -1.0

BELIEF_STATE: Uniform TigerLeft:0.5 TigerRight:0.5
BELIEF_STATE: AlmostSureLeft TigerLeft:0.95 TigerRight:0.05

DISCOUNT: 0.95
SOLVER: pbvi

VERIFY: pi(Uniform) == Listen
VERIFY: pi(AlmostSureLeft) == OpenLeft
```

III. COMPILATION PIPELINE

The compiler processes input through five mandatory phases, plus an optional HMM-bridge phase if requested.

Phase 0, Preprocessor. Expands GRID: macros into ordinary STATE / TRANSITION / REWARD lines, which the subsequent phases process uniformly.

Phase 1, Parser. A hand-written line-and-keyword recursive descent reads each declaration and populates a flat MDP_AST struct. The grammar is regular at file level and LL(1) per line; no separate tokenizer is required. Errors are reported with line numbers and stop the pipeline.

HMM bridge phase. If the file declares BRIDGE: hmm -> mdp, the bridge runs Baum-Welch on the referenced CSV, injects hidden states as MDP states, and adds the learned transition matrix as TRANSITION: rows. User-specified transitions for the same triples are preserved.

Phase 2, Validator. 25 named semantic checks read the AST and accumulate errors. Examples include: $\sum_{s'} T(s, a, s') = 1$ for every (s, a) pair within 10^{-9} (Check 5); $\sum_o O(s', a, o) = 1$ for every (s', a) (Check 15); $\sum_s b_0(s) = 1$ for the initial belief (Check 17). Each check has positive and negative unit tests in the suite.

Phase 3, Solver. The dispatcher consults ast.solver_mode and invokes one of four backends (Section IV), all returning a uniform SolverResult.

Phase 4, Verifier. Each `VERIFY`: assertion is parsed and evaluated. The process exits with status 1 if any assertion fails, 0 otherwise.

IV. SOLVER BACKENDS

A. Value iteration

We iterate the Bellman optimality operator

$$(T^*V)(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} T(s, a, s') V(s') \right] \quad (1)$$

from $V_0 \equiv 0$ until $\|V_{k+1} - V_k\|_\infty < \theta = 10^{-9}$. T^* is a γ -contraction in sup-norm, so by the Banach fixed-point theorem the iteration converges geometrically with rate γ . Per-iteration cost is $O(|T|)$ where $|T|$ is the number of declared transition rows; iteration count is $O(\log \theta / \log \gamma)$.

B. Policy iteration

PI alternates two steps. *Evaluation*: solve the linear system $(I - \gamma P^\pi)V^\pi = R^\pi$ where P^π is the $|\mathcal{S}| \times |\mathcal{S}|$ matrix of transitions under π . *Improvement*: compute the greedy policy with respect to V^π . Termination is detected when the policy stops changing.

The evaluation step uses Gaussian elimination with partial pivoting. Without pivoting, ill-conditioned matrices (small self-transition probabilities, or γ approaching 1) produced numerically meaningless solutions during development. The pivoting addition was approximately fifteen lines of code and eliminated a class of bugs.

C. Q-learning

Model-free Q-learning with ϵ -greedy exploration is included to verify that the same AST is solvable by both model-based and model-free methods. The step-size schedule $\alpha_t = 1/(1 + N(s, a))$ satisfies the Robbins-Monro conditions; ϵ decays linearly from 1.0 to 0.05 over 25,000 steps. On the 4×3 grid-world after 50,000 episodes, the recovered policy agrees with π^* on at least 8 of 11 states.

D. Point-based value iteration

For POMDPs, we implement PBVI [4]. The value function on $\Delta(\mathcal{S})$ is represented as the upper envelope of a set Γ of alpha-vectors. The backup at a belief point b proceeds in three stages:

- 1) For each action a and observation o , find the previous alpha-vector to follow:

$$\alpha_{a,o}^* = \arg \max_{\alpha \in \Gamma_{\text{prev}}} \sum_s b(s) \sum_{s'} T(s, a, s') O(s', a, o) \alpha(s').$$

- 2) Construct the action-conditional vector:

$$\alpha_a(s) = R(s, a) + \gamma \sum_o \sum_{s'} T(s, a, s') O(s', a, o) \alpha_{a,o}^*(s').$$

- 3) Return α_{a^*} where $a^* = \arg \max_a \langle b, \alpha_a \rangle$.

Witness pruning. We retain α if and only if there exists a belief b in the current belief set B at which α is strictly best. An earlier implementation used general dominance (that

is, α dominated iff some α' exceeds α on every state) and over-pruned: the Tiger problem collapsed to a single alpha-vector and produced an incorrect policy at the uniform belief. Witness pruning yields the expected 3 to 5 useful vectors and the correct policy. The fix is recorded as a regression test that asserts $|\Gamma| \in [3, 7]$ on Tiger.

V. HMM-MDP BRIDGING

The Baum-Welch fitter is implemented in pure C++17 with Rabiner's scaling discipline. For each timestep, the scaling factor $c_t = 1/\sum_i \alpha_t(i)$ normalizes the forward variables; the log-likelihood is recovered as $\log P(O | \lambda) = -\sum_t \log c_t$. Without scaling, forward variables underflow to zero for sequences beyond approximately 50 steps with Gaussian emissions of moderate variance, which breaks the M-step. A dedicated test verifies the fitter on a 500-step synthetic sequence.

Listing 2 shows the portfolio model. The user declares four hidden states and points the bridge at a CSV; the fitted transition matrix is plumbed into the MDP and the file is validated and solved as a single artifact.

Listing 2. HMM-bridged portfolio optimizer (excerpt).

```
HMM_STATES: Bull Bear Sideways Crisis
EMISSION_TYPE: gaussian
FIT_HMM: portfolio_returns.csv iterations=200
HMM_SEED: 42

BRIDGE: hmm -> mdp

ACTION: AggressiveEquity
ACTION: ModerateEquity
ACTION: Defensive
ACTION: CashAndGold

# Sharpe-calibrated rewards (excerpt):
ACTION_REWARD: Bull AggressiveEquity 0.85
ACTION_REWARD: Bear Defensive 0.10
ACTION_REWARD: Crisis CashAndGold 0.08

DISCOUNT: 0.92
SOLVER: pi

VERIFY: pi(Bull) == AggressiveEquity
VERIFY: pi(Bear) == Defensive
VERIFY: pi(Crisis) == CashAndGold
VERIFY: V(Bull) > V(Bear)
VERIFY: V(Crisis) < V(Sideways)
```

The portfolio sub-system fetches NSE/BSE equity prices via `yfinance` (15-minute local cache, seed-price fallback when offline), fits the HMM on monthly returns, calibrates per-regime Sharpe ratios from Viterbi-decoded segments, emits the `.mdp`, invokes the compiler with `--json`, and proposes a portfolio allocation. A dashboard, implemented as a single HTML file with no npm dependencies, renders the regime history, the policy boundary heat-map over belief space, the allocation donut, and the `VERIFY` status panel.

VI. THE AUTOPSY ENGINE

The autopsy engine diagnoses six classes of reward mis-specification statically, before or independently of any training process.

A. Six failure classes

Reward myopia. The effective planning horizon $H_{\text{eff}} = \log \varepsilon / \log \gamma$ with $\varepsilon = 10^{-3}$. BFS computes the minimum distance d_{goal} from each state to any positive-reward state. If $d_{\text{goal}} > H_{\text{eff}}$, the agent is structurally blind to the goal. The recommended fix is the minimum $\gamma' = \exp(\log \varepsilon / d_{\text{goal}})$.

Reward hacking. The optimal-policy graph (each state's outgoing edge follows π^* , mode-collapsed) is analyzed via Tarjan's strongly-connected-components algorithm. For each non-singleton SCC C with reward sum r_C , the cycle value under optimal play is $V_{\text{cycle}} = r_C / (1 - \gamma^{|C|})$. The detector fires if $V_{\text{cycle}} > 1.05 \cdot V^*$ at the best state outside C . Absorbing terminal goals are excluded from cycle analysis.

Discount cliff. The optimal policy is computed at 200 points across $\gamma \in [0.01, 0.999]$; cliffs are values at which $\pi(s)$ flips between adjacent grid points for any state s . The detector reports the cliff nearest the user's γ along with the precise flip.

Magnitude imbalance. For each non-zero reward term, the contribution $c_i = |r_i| / \sum_j |r_j|$ is computed. Terms below 0.001 are flagged as effectively ignored; the recommended scale-up is the minimum to reach 5% contribution.

Dead-state detection. Reverse BFS from the set $\{s : R(s) > 0 \vee \exists a. R(s, a) > 0\}$ identifies states that no policy can reach a reward from.

Fragility analysis. For each $(s, \pi^*(s))$ pair and each perturbable parameter (every reward and γ), binary search finds the smallest $|\delta|$ that flips $\pi^*(s)$. The report ranks the top-5 most fragile decisions and computes a robustness score.

B. REPAIR command

Given a target assertion such as `pi(Start) == MoveRight`, the `--repair` mode parses the assertion, checks if it already holds, and otherwise binary-searches each candidate parameter for the smallest perturbation that satisfies it. The proposals are returned ranked by magnitude. Multi-parameter joint repair is noted as future work; the current implementation handles only single-parameter searches.

C. Backwards solver

A separate binary, `mdp_autopsy`, consumes an execution log ending in a `FAILURE` marker and produces a graded verdict. Empirical transition frequencies $\hat{T}(s, a, s')$ are computed, skipping transitions whose source is an absorbing state in the model (this filters mission-reset events such as physical reset after collision). The KL divergence between empirical and model transitions, with additive smoothing $\varepsilon = 10^{-10}$, ranks the suspect pairs.

The backwards solver itself answers: what is the smallest perturbation ΔT along the empirical direction at which a `VERIFY` assertion fails? We parameterize $T_\alpha = (1 - \alpha)T_{\text{model}} + \alpha\hat{T}$ and binary-search $\alpha \in [0, 1]$ for the smallest value at which any assertion fails.

Scope statement. A free L^1 minimization over the constrained simplex would require a QP solver, which would violate our zero-dependency constraint. Linear search along

the empirical direction is an honest approximation: the empirical data is the direction in which the world disagrees with the model, which is exactly the axis the autopsy is meant to investigate. This limitation is reported in the autopsy output verbatim.

The verdict ladder is shown in Table I.

TABLE I
VERDICT LADDER FOR `MDP_AUTOPSY`.

Worst factor	Verdict	Exit code
$< 2\times$	MODEL_ACCURATE	0
$2\times$ to $10\times$	MODEL_OPTIMISTIC	1
$\geq 10\times$	MODEL_DANGEROUS	2

VII. EXPERIMENTAL RESULTS

A. Test suite

The full test suite comprises 174 tests and 297 assertions, distributed as in Table II. The complete suite executes in roughly one second.

TABLE II
TEST SUITE DISTRIBUTION ACROSS PROJECT PHASES.

Group	Tests	Assertions
Baseline (parser / validator / VI / PI / QL)	94	142
POMDP / PBVI / belief update	21	47
HMM / Baum-Welch / bridge	15	36
Autopsy engine (6 classes plus REPAIR)	28	43
Backwards solver / log parser	16	29
Total	174	297

B. Tiger POMDP

On Tiger ($|\mathcal{S}| = 2$, $|\mathcal{A}| = 3$, $|\Omega| = 2$, $\gamma = 0.95$), PBVI converges in roughly 100 backups, yielding 4 useful alpha-vectors after witness pruning. Both `VERIFY` assertions pass.

C. Portfolio model

On the HMM-bridged portfolio model (4 hidden states, 4 actions, 60 months of Nifty returns, $\gamma = 0.92$, PI solver), all five `VERIFY` assertions pass. The fitted regime persistence matrix has diagonal entries $\{0.83, 0.79, 0.71, 0.62\}$, broadly consistent with macroscopic Indian-market behavior over the training window.

D. Autopsy detections

On crafted demonstrations:

- Myopic corridor (9 states, $\gamma = 0.30$, asymmetric action cost): `RewardMyopia ERROR` with $H_{\text{eff}} = 5.7$ and $d_{\text{goal}} = 8$; `DiscountCliff WARNING` at $\gamma^* = 0.27$.
- Reward hacking demo (cycle $S_2 \leftrightarrow S_3$ yielding +12/step, one-shot goal +50): `RewardHacking ERROR` with $V_{\text{cycle}} = 246.15$ exceeding V^* outside the cycle, which was 228.00.
- Dead-state demo (disconnected trap cycle): `DeadState ERROR` with both trap states identified.
- Repair demo (3-state corridor, $\gamma = 0.15$, move cost -5): the repair search finds 6 fixes; the minimum is γ from 0.15 to 0.218, that is, $\Delta = 0.068$.

E. Backwards-solver verdicts

Table III summarizes four demonstration logs.

TABLE III
MDP_AUTOPSY VERDICTS ON FOUR DEMONSTRATIONS.

Log	Verdict	Backwards solver
medical_clean.log	ACCURATE	no failure to back-solve
medical_run.log	OPTIMISTIC	$\alpha=0.04$, $\ \Delta T\ _1=0.04$
robot_run.log	DANGEROUS	VERIFY robust under empirical
portfolio_run.log	DANGEROUS	$\alpha=0.32$, $\ \Delta T\ _1=0.76$

The robot demonstration is worth a comment. The empirical (NearObstacle, Advance) $\rightarrow$ Collision transition occurred at 0.25 vs. the model’s 0.02, a factor of 12.5x. Yet the backwards solver reports that no perturbation along the empirical direction breaks any VERIFY assertion. Under empirical transitions, the optimal action at NearObstacle is SlowDown, which is actually safer in reality than in the model. The diagnostic message is therefore that the engineer’s safety properties were correctly designed, and the run-time failure occurred because the deployed policy ignored the model and selected Advance repeatedly, not because the model’s safety specification was inadequate. We find this kind of distinction (design-level vs. deployment-level failure) useful and not always easy to make by hand.

VIII. RELATED WORK

POMDP solvers. APPL/SARSOP [6] and HSVI [5] implement modern point-based solvers focused on performance. The PBVI implementation in MDP-DSL is a teaching-grade implementation appropriate to the project’s scope and not intended to compete on solver throughput.

Probabilistic verification. PRISM [7] and STORM [8] verify temporal-logic properties of Markov chains and MDPs. They accept rich property languages but require the policy to be specified or enumerated; they do not themselves solve Bellman optimality. MDP-DSL goes the other way: solve the MDP, then verify properties of the optimal policy.

RL specification gaming. Krakovna et al. [1] catalog 60+ examples of agents exploiting reward specifications in unintended ways. The autopsy engine here is a small tooling response: several of these failures are detectable statically from the reward function alone (cycles, dead states, discount cliffs).

DSLs for sequential decisions. POMDP file formats (such as Cassandra’s .pomdp format) exist as serialization formats for solvers, but typically lack semantic checking, verification, or HMM bridging. MDP-DSL adds these in a single artifact.

IX. ENGINEERING CONSTRAINTS AND HONEST SCOPE

The implementation enforces five constraints. First, no external C++ dependencies (STL only). Second, single-translation-unit compilation for the main binary. Third, byte-identical backward compatibility across all language extensions. Fourth, exit-code semantics suitable for CI. Fifth, total source under roughly 13,000 lines.

Three scope limitations are documented explicitly in the implementation and report output.

Backwards solver approximation. The Phase 3B backwards solver searches along the empirical direction rather than freely minimizing $\|\Delta T\|_1$ over the constrained simplex (which would require a QP solver). The implementation reports this approximation explicitly.

REPAIR single-parameter only. The repair search perturbs one parameter at a time. Multi-parameter joint repair is future work.

Underestimation factor cap. When the model assigns $p < 10^{-6}$ to an empirically-observed transition, the raw ratio is mathematically unbounded. We cap the displayed factor at $1000\times$ to keep reports informative; the KL ranking remains well-defined via additive smoothing.

We argue that explicit scope statements of this kind are more credible in tooling intended for verification and diagnosis than implicit claims of optimality.

X. AUTHOR’S CONTRIBUTION STATEMENT

The project was specified, architected, and led by the author. The v2.0 baseline (value iteration, policy iteration, Q-learning, the VERIFY block, the GRID macro, and the original five-phase pipeline) was completed in the spring 2026 term as the author’s DA 221M Artificial Intelligence minor-project term work under the supervision of Prof. Shyamanta M. Hazarika at IIT Guwahati.

The v3.0 extensions described in this paper (the POMDP/PBVI extension, the HMM-MDP bridge, the portfolio subsystem, the six-class autopsy engine, and the backwards-solver binary) are post-coursework work undertaken by the author independent of the supervised project. Implementation of these extensions used AI-assisted code generation under the author’s direction. Every architectural choice, scope constraint, algorithm selection, and verification semantic was authored by the candidate. AI assistance was used at the implementation level. This is disclosed for accuracy.

XI. CONCLUSION AND FUTURE WORK

MDP-DSL is a small contribution toward a compiler-first approach to sequential decision-making. The 25-rule validator catches model misspecification before any solver runs; the VERIFY block turns model assumptions into compile-time assertions; the autopsy engine diagnoses six classes of reward pathology and proposes repairs.

The longer-term direction the author intends to pursue is to grow the project from a compiler into an integrated development environment for sequential-decision specification: a workspace where practitioners can write a model, see solver and autopsy output interactively, iterate on the specification, and version it as a first-class artifact. Smaller future directions include inverse MDP compilation (algebraic recovery of MDP structure from observed policies), discount-cliff cartography, and extending the autopsy machinery beyond MDPs to other forms of objective specification.

The implementation is open-source under the MIT license. The build is reproducible with a single `g++` invocation and no external dependencies. Issues, bug reports, and contributions are welcome.

ACKNOWLEDGMENTS

The author thanks Prof. Shyamanta M. Hazarika (IIT Guwahati) for supervising the v2.0 baseline as part of the DA 221M course.

REFERENCES

- [1] V. Krakovna et al., “Specification gaming examples in AI,” DeepMind Safety Research Blog, 2020.
- [2] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, John Wiley & Sons, 2014.
- [3] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [4] J. Pineau, G. Gordon, and S. Thrun, “Point-based value iteration: An anytime algorithm for POMDPs,” in *IJCAI*, 2003, pp. 1025–1032.
- [5] T. Smith and R. Simmons, “Heuristic search value iteration for POMDPs,” in *UAI*, 2004, pp. 520–527.
- [6] H. Kurniawati, D. Hsu, and W. S. Lee, “SARSOP: Efficient point-based POMDP planning by approximating optimally reachable belief spaces,” in *Robotics: Science and Systems*, 2008.
- [7] M. Kwiatkowska, G. Norman, and D. Parker, “PRISM 4.0: Verification of probabilistic real-time systems,” in *CAV*, 2011, pp. 585–591.
- [8] C. Dehnert, S. Junges, J.-P. Katoen, and M. Volk, “A storm is coming: A modern probabilistic model checker,” in *CAV*, 2017, pp. 592–600.
- [9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [10] E. J. Sondik, “The optimal control of partially observable Markov processes,” Ph.D. dissertation, Stanford University, 1971.